

Master in Big Data Analytics

2026-2027

TECHNOLOGICAL FUNDAMENTALS IN THE BIG DATA WORLD

Lab 1: K-Means Parallelization in Python

Authors

NIA	Surname(s)	Name
100592076	Amón Álvarez	Juan Guillermo
100604007	García Campos	Diego Froilán
100601719	García García	Miguel
100587804	Vázquez Estrada	Alfredo

1. Introduction

1.1 Goal and Scope

The goal of this laboratory is to parallelize a Python program and to analyse the performance gain obtained with two of the parallel programming models available in Python: multiprocessing and multithreading. As the base line for the parallelization we will use the k -Means clustering algorithm on a synthetic dataset of two million proteins. However, the object of study is analysing the quality of the parallelization rather than the clustering itself; therefore, the report focuses on how the program is decomposed, which parts can be parallelized, how each section behaves, and how far the measured speedups are from the theoretical limits.

1.2 Why k -means can be parallelized

The bottleneck of data analysis nowadays is processing big volumes of data in a timely and scalable fashion. K -means clustering on two million points is a reasonably good toy example for managing relatively big workloads.

The k -Means Clustering Algorithm. k -Means is an unsupervised machine learning algorithm [1] that helps group data points into clusters based on their inherent similarity. In other words, the algorithm categorizes the data points into k groups or clusters of similarity. To calculate that similarity we use the Euclidean distance as a measurement:

$$d(x_1, x_2) = \sqrt{(x_2 - x_1)^2}$$

The algorithm works as follows:

1. **Initialization:** k random points of the dataset are taken as initial centroids.
2. **Assignment:** each point is assigned to its nearest centroid, forming clusters.
3. **Update:** the centroid of each cluster is recalculated by averaging the points within it.
4. **Convergence:** steps 2–3 are repeated until the centroids stop moving or a maximum number of iterations is reached.

Selecting the right number of clusters k is important for meaningful segmentation. For a given k , the algorithm looks for the k centroids that minimize the within-cluster sum of squares (WCSS):

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Choosing k: the elbow method. To find the optimal number of clusters k we compute WCSS by evaluating a range of $k \in [1, 15]$ using the Elbow Method. The criterion used to find the "elbow" of the curve mathematically is the maximum perpendicular distance. The algorithm constructs a secant line connecting the first point $(1, WCSS_1)$ and the last point $(15, WCSS_{15})$. Then, it calculates the perpendicular distance from every point on the inertia curve to this reference line. The k value associated with the maximum distance is automatically selected as the optimal number of clusters.

Parallelization. The k -Means Algorithm, particularly when combined with the Elbow method, presents a partially “embarrassingly parallel” workload structure. Evaluating the Within-Cluster Sum of Squares (WCSS) for multiple candidate values of k requires running the entire clustering process independently for each value. Since the computation for $k = 2$ has no dependency on the results of $k = 3$, these executions can be perfectly distributed.

1.3 Dataset

The data are generated with `proteins-generator.py`, which writes `proteins.csv` with the fields `protid`, `enzyme`, `hydrofob` and `sequence`. All measurements use a dataset of 2,000,000 proteins (≈ 269 MB) generated with seed 123. Clustering uses the two numerical fields, `enzyme` and `hydrofob`, and the `sequence` is only used through its length.

1.4 Explanation of the class `KMeansCustom`

In order to make the code as modular as possible and facilitate parallelization, we employ a custom Python class called `KMeansCustom` (located in `kmeans_scratch.py`). This approach allows us to reuse the exact same algorithm across the serial, multiprocessing, and threading implementations, ensuring perfectly comparable execution times.

The class initializes with key hyperparameters: the number of clusters (k), maximum iterations (`max_iter=300`), a convergence tolerance (`tol=1e-4`), and a seed for a consistent random initialization.

Internally, the `fit()` method is heavily optimized using NumPy vectorization [3] to bypass slow native Python loops. The most significant optimization is the calculation of the squared Euclidean distances. Instead of computing the distance naively, the algorithm mathematically expands the formula to $(x - \mu)^2 = x^2 + \mu^2 - 2x\mu$. This allows the program to calculate the squared norms of all data points (x^2) just once before entering the optimization loop. Inside the loop, the distances are computed efficiently using matrix multiplication (`X @ self.centroids_.T`), drastically reducing the computational overhead.

Furthermore, the reliance on NumPy is a strategic choice for parallelization. In Python, the Global Interpreter Lock (GIL) typically prevents multiple threads from executing bytecode simultaneously, bottlenecking CPU-bound multithreaded applications [4]. To accurately measure the performance gain of our high-level parallelization—and prevent NumPy from interfering with its own internal multithreading—we explicitly restrict the BLAS backend to a single thread using the `threadpool_limits` function from the `threadpoolctl` library [5].

Additionally, the class includes robust error handling: if a cluster becomes empty during the assignment phase, the algorithm retains its previous centroid position rather than crashing. Finally, an early-stopping convergence check breaks the loop if the absolute change in inertia between iterations falls below the tolerance.

2. Experimental setup

2.1 Hardware and software

To evaluate the performance of our parallel implementations across different hardware architectures, the experiments were conducted on the personal computers of all four team members (Table 1). All tests were executed with the laptops plugged into AC power and with no heavy background tasks running to avoid CPU throttling. All of the laptops executed the same code, using the same dataset (2,000,000 proteins generated with seed 123) and the same configurations: the serial version, the multiprocessing version with 15 worker processes (one per value of k), and the threaded version with 4, 8, 12 and 16 threads.

All algorithms were executed using Python 3, with data manipulation handled by Pandas and mathematical operations vectorized through NumPy.

Table 1. Specifications of each laptop.

	Laptop 1	Laptop 2	Laptop 3	Laptop 4
Physical cores	8	20	10	8
Logical cores	16	20	16	16
Base frequency	3.2 GHz	2.4 GHz	1.9 GHz	3.3 GHz
RAM	16 GB	32 GB	32 GB	16 GB
Operating system	Windows 11	Windows 11	Windows 11	Windows 11

2.2 Measurement methodology

Execution times are measured inside the programs with `time.time()`, separately for each stage (preprocessing, elbow method, selection of the optimal k , final clustering and sequence analysis). The total time goes from the start of the program until the sequence analysis has finished; the plots are generated afterwards and are not included.

2.3 Performance metrics

We compare the different implementations of the algorithm using the following metrics [2]:

- **Speedup:** how many times faster a parallel version is than the serial one, or other parallel implementations:

$$S = \frac{T(x)}{T(y)},$$

where $T(x)$ = computational time of the slower version and $T(y)$ = computational time of the faster version.

- To see the maximum speed that can be theoretically achieved, we use the Amdahl's Law:

$$S = \frac{1}{(1-P) + \frac{P}{N}},$$

where P = proportion of the execution time that is done in parallel and N = the number of available computing cores.

3. Serial baseline

3.1 Explanation of the file lab1-proteins-serial.py

The `lab1-proteins-serial.py` script serves as the baseline for our study. It executes the entire k -Means clustering pipeline sequentially, using a single core. The logic of the script is structured in the following sequential blocks:

1. **Loading and preprocessing:** The program starts a global timer and reads the `proteins.csv` dataset using the `pandas` library. It computes the sequence length (`seq_length`), extracts the numerical variables `enzyme` and `hydrofob` into a NumPy array, and applies Min-Max normalization to scale both features into a $[0, 1]$ range. This transformation, computed as $x' = \frac{x - \min}{\max - \min}$, gives both features the same weight and

stores them as floating-point numbers, so the centroids are not truncated to integers during the clustering process.

2. **Elbow method** (`compute_wcss`): runs `KMeansCustom` for every k from 1 to 15 and stores the WCSS of each run.
3. **Optimal K** (`find_optimal_k_elbow`): selects the elbow of the curve as described in Section 1.2.
4. **Final clustering**: With the optimal k identified, the program runs the K-Means algorithm one last time to fit the data and assigns the resulting cluster labels to the original dataset.
5. **Cluster analysis**: finds the cluster that contains the protein with the longest sequence (ties are broken by the highest hydrofob) and computes its average sequence length.

The timer stops after stage 5. The three plots (elbow graph, clusters with centroids and heat map of the centroids) are generated afterwards, so their rendering time does not affect the measurements.

3.2 Program workflow and computational time

Before implementing any parallelization strategy, it is mandatory to profile the serial execution to identify the performance bottlenecks. We recorded the execution time of each independent stage to determine the parallelizable fraction of the code (P) required for Amdahl's Law.

Table 2: percentage of the total serial runtime spent in each stage. Laptop 2.

Stage	Time (s)	% of total	Parallelizable?
Loading and preprocessing	2.20	1.4 %	I/O-bound
Elbow method (15 k-means runs)	156.08	95.9 %	Yes: independent runs
Selection of the optimal k	0.005	0.0 %	Negligible
Final clustering	4.00	2.5 %	Only a single run
Cluster analysis	0.53	0.3 %	Negligible
Total	162.81	100 %	

This breakdown clearly reveals that the elbow method takes around 95% of the execution time on all the computers and consists of 15 independent tasks, so it is the target of the parallelization. The remaining few percent, dominated by reading the CSV file and by the final clustering, stays serial and limits the maximum speedup according to Amdahl's law. The cost of the tasks is not uniform: it depends on k and, above all, on the number of iterations that each run needs to converge, which matters for load balancing.

4. Parallel implementation

4.1 Parallelization methodology

Decomposition. Each k -means run of the elbow method is an independent task. This is task parallelism with the task-farming (master/worker) pattern: as in hyperparameter tuning, each worker runs the same algorithm with a different parameter (k), and the master collects the results when all the tasks have finished. The tasks are coarse-grained: each one is a complete k -means run of several seconds.

Assignment. The tasks are distributed among the workers. In the multiprocessing version, `Pool.starmap` hands the tasks to the processes of a pool; in the threaded version, the values of k are split into contiguous blocks, one block per thread.

Orchestration. The tasks do not communicate with each other. Each worker only needs the dataset and returns the WCSS of its run, not the labels of the two million points. With multiprocessing, each process has its own address space, so the dataset is copied to the processes. With threads, the address space is shared: the dataset is not copied, and each thread writes its results into its own positions of a shared list, so no critical section is needed.

Mapping. The operating system maps the processes or threads onto the cores. The computers used are shared-memory multicore machines. Laptops 1, 3 and 4 have more logical than physical cores because of simultaneous multithreading (SMT) [6].

4.2 Multiprocessing implementation

Python's multiprocessing module bypasses the Global Interpreter Lock (GIL) by spawning independent memory processes. Since the serial time breakdown revealed that the inertia (WCSS) calculation consumes over 95% of the total execution time, we parallelized the workload at the macro-level. Instead of parallelizing the internal K-Means algorithm, the script evaluates each candidate value of k in a completely separate process.

We implemented this using the `multiprocessing.Pool` class alongside the `starmap` method [7], mapping the `compute_wcss_for_k` function over $k = 1, \dots, 15$. The pool has `min(cpu_count(), 15)` processes, so 15 processes are used on all the computers, one per task; since the tasks are handed out as processes become free, the pool balances the load automatically. Additionally, `starmap` returns the results in the correct order to build the WCSS list, and blocks until all the tasks have finished.

However, the multiprocessing module relies on independent memory spaces, so processes cannot easily share variables. In other words, the entire dataset must be serialized and copied into the memory space of every worker process before the computation begins. This data transmission introduces significant overhead, so parallelization is only worthwhile for heavy, time-consuming tasks that offset this initial copying penalty. In addition, on Windows the processes are started with the spawn method, so each worker imports the program again; the guard `if __name__ == "__main__":` prevents the main program from running in every worker.

Alternative implementations (parallelizing centroid updates inside the *K*-Means loop) were discarded because the severe overhead of repeatedly creating and destroying process pools on every iteration vastly outweighed any computational gains.

4.3 Threading implementation

While the multiprocessing approach creates isolated memory spaces, the threading module spawns multiple threads that share the same memory space. To evaluate the performance, we implemented a multithreaded version targeting the same WCSS computation.

To distribute the workload among the specified number of threads, we used `numpy.array_split` to divide the range of indices associated with our candidate *k* values into balanced chunks. A target worker function is then executed by each `threading.Thread`, taking its assigned chunk of indices as an argument. As each thread evaluates the `KMeansCustom` for a specific *k*, it writes the resulting inertia directly into its corresponding index within the shared `wcss_list`. Finally, the main thread calls `.join()` on all threads to ensure every calculation is completed before proceeding to the Elbow method logic.

Regarding the Global Interpreter Lock (GIL) [4], it acts as a mutex that protects access to Python objects and prevents multiple threads from executing Python bytecodes simultaneously. Typically, this makes multithreading useless for CPU-bound tasks like mathematical clustering. However, our custom *K*-Means implementation relies heavily on NumPy for its core operations (specifically for the vectorized distance calculations). Since NumPy explicitly releases the GIL when performing intensive mathematical computations, our worker threads are able to achieve genuine concurrent execution at the hardware level during the heaviest parts of the algorithm, and only compete for the GIL in the short Python code between NumPy calls.

5. Results

5.1 Clustering results

All the versions obtain exactly the same result on all the computers: the elbow method selects $k = 3$, and the cluster that contains the longest protein (id 194638, length 263) has an average sequence length of 126.06. Figures 1-3 shows the plots required by the lab.

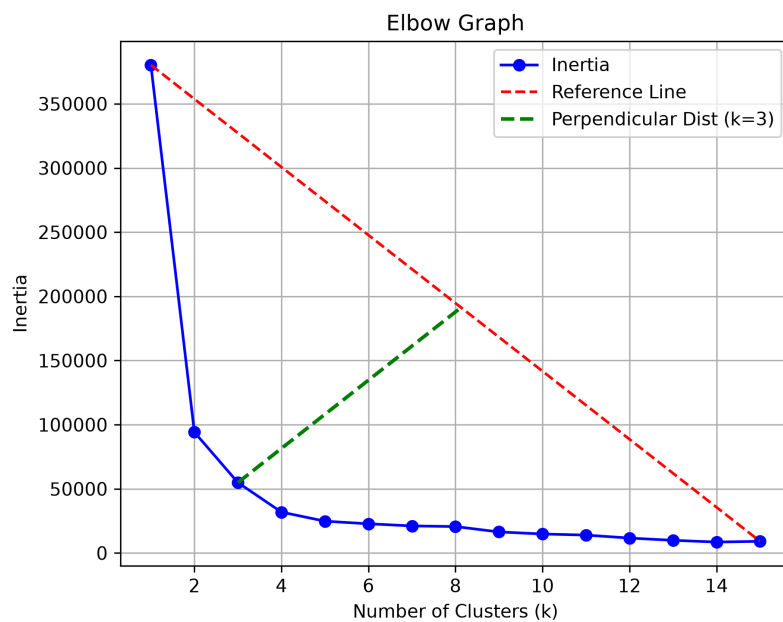


Figure 1. Elbow graph

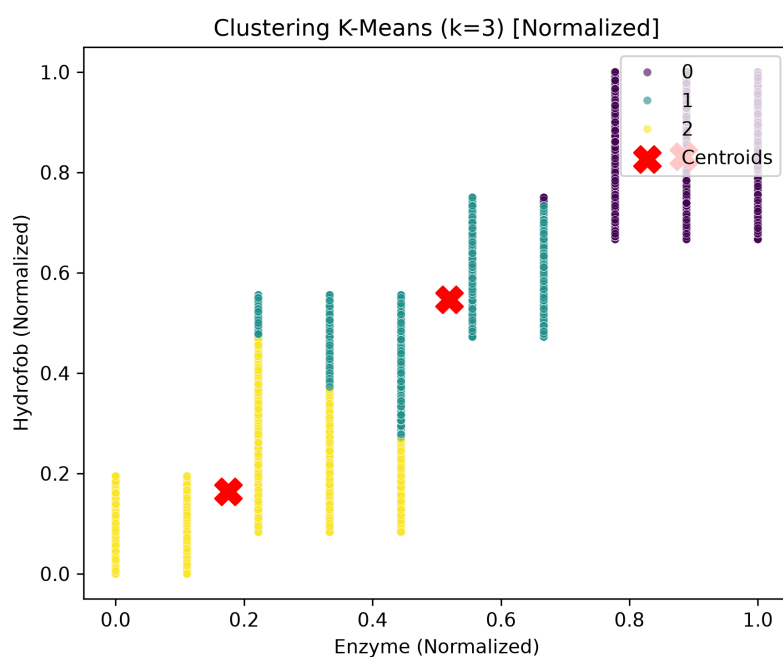


Figure 2. Clusters with their centroids. Normalized features.

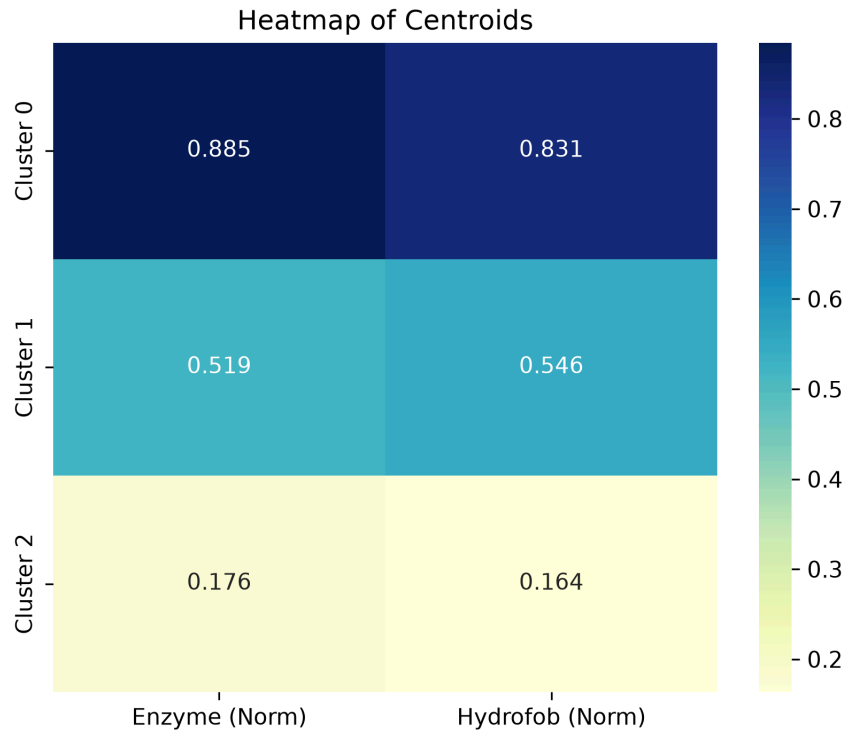


Figure 3. Heat map of the centroids. Normalized features.

5.2 Execution times and speedup on Laptop 2

Table 3 shows the time breakdown of every version on Laptop 2, the computer with the most cores.

Table 3. Time per stage (s) and speedups on the laptop 2 (20 cores).

Version	Workers	Preproc.	Elbow (inertia)	Final fit	Total	Speedup	Elbow Speedup
Serial	1	2.20	156.08	4.00	162.81	1.00	1.00
Multiprocessing	15	3.67	34.54	4.47	43.13	3.78	4.52
Threading	4	3.48	51.75	3.87	59.56	2.73	3.02
Threading	8	3.71	45.56	4.92	54.64	2.98	3.43
Threading	12	3.54	27.19	4.19	35.37	4.60	5.74
Threading	15	3.30	31.63	4.74	40.10	4.06	4.93

Both parallel models reduce the execution time substantially. Multiprocessing runs 3.78 times faster than the serial version, and threading reaches its best result with 12 threads (4.60). The serial stages keep the same duration in all the versions (about 4 s of final clustering and 2–4 s of preprocessing). So while the elbow method shrinks, their share of the total grows from about 4 % in the serial version to 23 % with 12 threads: exactly the effect described by Amdahl's law.

5.3 Speedup on all computers

Table 4 compares the speedups of the whole program on the four computers, and Figure 4 shows them as a function of the number of workers.

Table 4. Speedup of the whole program on each computer.

Version	Laptop 1 (8 c.)	Laptop 2 (20 c.)	Laptop 3 (10 c.)	Laptop 4 (8c.)
Serial time (s)	119.12	162.81	145.50	183.76
Multiprocessing, 15 proc.	2.05	3.78	4.45	2.86
Threading, 4 threads	1.89	2.73	2.79	2.13
Threading, 8 threads	2.33	2.98	2.84	2.42
Threading, 12 threads	2.43	4.60	3.24	3.82
Threading, 15 threads	2.31	4.06	3.58	3.91
Amdahl max N = n ^o of cores	5.47 (N=8)	11.21 (N=20)	6.32 (N=10)	5.33 (N=8)
Amdahl max N = 15 tasks	7.79	9.50	8.33	7.49

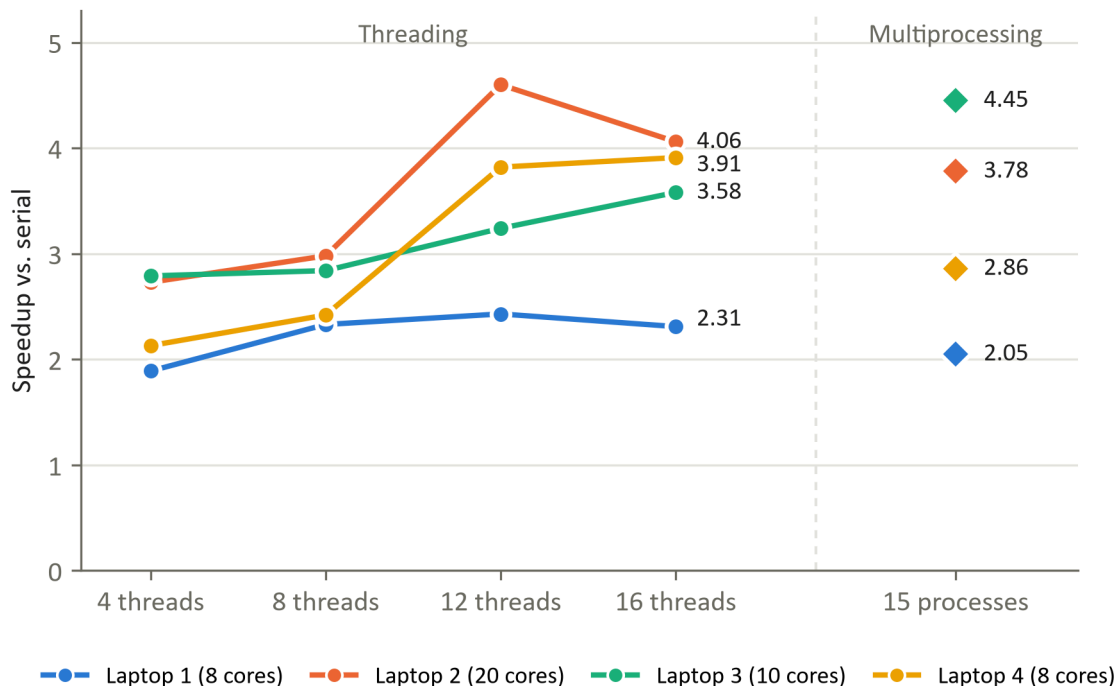


Figure 4. Speedup as a function of the number of workers. Lines: threading.

Diamonds: multiprocessing with 15 processes.

Three observations stand out. First, the speedup grows with the number of cores: it stays around 2 on Laptop 1 (8 physical cores) and reaches 4.6 on Laptop 2 (20 cores). Second, threading is competitive

with multiprocessing, since it is faster on Laptops 1, 2, and 4 (12 threads on Laptop 1: 2.43 vs 2.05; 12 threads on Laptop 2: 4.60 vs 3.78; 16 threads on Laptop 4: 3.91 vs 2.86), while on Laptop 3 multiprocessing remains the best (4.45 vs 3.58). Third, the speedup of threading does not grow steadily with the number of threads: on Laptop 2 it is higher with 12 threads than with 16, and on Laptop 1 it barely changes beyond 8 threads.

6. Discussion

Multiprocessing versus threading. Multiprocessing gives each process its own interpreter, so the values of k run with true parallelism. In exchange, it pays for creating 15 processes, importing the program again in each of them (spawn) and copying the dataset to every worker. These costs are small compared with a complete k -means run on two million points, which is why multiprocessing achieves a solid speedup on all the computers. Threading avoids them, since all the threads share the dataset in memory. In this program, almost all the time is spent inside NumPy operations that explicitly release the GIL, and the threads only compete for the GIL in the short Python code between those calls. As a result, threading matches or beats multiprocessing on three of the four computers (12 threads: 4.60 vs 3.78 on Laptop 2; 2.43 vs 2.05 on Laptop 1; and 16 threads: 3.91 vs 2.86 on Laptop 4).

Why threading does not scale significantly beyond 12 threads. The threaded version distributes the work statically: each thread receives a contiguous block of values of k . The blocks have the same number of tasks, but not the same amount of work, because the cost of a k -means run depends on how many iterations it needs to converge rather than on k . On Laptop 1, for example, the run for $k = 15$ alone takes 17.1 s out of 120 s of total work. With 12 or more threads, the elbow method can never finish before that single run, so adding threads beyond 12 brings no gain and only increases the competition for the GIL and for memory. The multiprocessing pool, in contrast, hands the tasks to the processes as they become free. A dynamic assignment for the threads, or sorting the tasks from the most to the least expensive, would balance the load better.

Distance to the theoretical limit. Amdahl's law only accounts for the serial fraction of the program and assumes that the parallel part is divided perfectly among the workers. With $P \approx 0.96$, it predicts a speedup of 9.5 for 15 workers on Laptop 2, while the best measured value is 4.6. The law ignores the costs that parallelization adds: creating processes, copying data, load imbalance, the GIL and, above all, the competition for shared hardware. With two million points, every iteration streams arrays of tens of megabytes from RAM, so the workers compete for the same memory bandwidth and caches. On Laptop 1, moreover, 15 workers share only 8 physical cores, and the two logical cores of a physical

core (SMT) share its execution units, so the useful parallelism is closer to 8 than to 15. This explains why the speedup grows with the number of physical cores, from about 2.4 on Laptop 1 (8 cores) to 4.6 on Laptop 2 (20 cores).

7. Conclusions

The serial profile showed that the elbow method takes between 93% and 96% of the execution time on all the computers and consists of 15 independent k -means runs, which makes it a natural task farm. Parallelizing it with coarse-grained tasks, one complete run per value of k , proved to be the right decomposition: the finer-grained alternatives, which parallelized each iteration of k -means or the small stages of the program, were slower because the cost of creating processes and moving data dominated.

Both parallel models reduce the execution time considerably, with speedups of up to 3.78 with multiprocessing and 4.60 with threading on the 20-core computer. Contrary to the usual rule of thumb for CPU-bound Python code, threading is competitive with multiprocessing here: NumPy releases the GIL during the heavy operations, and threads avoid the cost of creating processes and copying the data.

All the measured speedups remain well below the bound given by Amdahl's law. The law only accounts for the serial fraction of the program, while the real limits come from the uneven cost of the tasks, the number of physical cores, the shared memory bandwidth and the costs added by the parallelization itself.

Finally, the lab taught us to measure before parallelizing and to choose the granularity of the tasks carefully. Possible improvements are a dynamic assignment of the values of k to the threads, ordering the tasks from the most to the least expensive, and testing larger datasets, where the weight of the serial part and of the overheads would be smaller.

Use of Generative AI

In the elaboration of this project we used Generative AI to aid us in the following tasks:

- Researching the k -means algorithm and implementing it in Python.
- Assistance in debugging the Python code.
- Finding ways to increase the speedup of our code by restricting the use of multithreading by NumPy's BLAS library.

The overall strategy and design of the project was done by us, and all generated code was reviewed by a human.

References

- [1] G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*. Springer, 2023, Section 12.4.1, pp. 521–525.
- [2] Course material. Technological Fundamentals in the Big Data World: Theory Slides and Lab 1 presentation. Universidad Carlos III de Madrid, 2026.
- [3] NumPy Developers. "NumPy Documentation". Available at: <https://numpy.org/doc/stable/> (accessed Oct. 5, 2026)
- [4] Real Python. "Speed Up Your Python Program With Concurrency". Available at: <https://realpython.com/python-concurrency/> (accessed Oct. 5, 2026)
- [5] threadpoolctl Developers. "threadpoolctl: Python helpers to limit the number of threads used in native libraries". Available at: <https://github.com/joblib/threadpoolctl> (accessed Oct. 5, 2026)
- [6] D. M. Tullsen, S. J. Eggers and H. M. Levy, "Simultaneous Multithreading: Maximizing On-Chip Parallelism," in *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*, pp. 392–403, 1995.
- [7] Python Software Foundation. "multiprocessing — Process-based parallelism" and "threading — Thread-based parallelism". Python 3 Documentation. Available at: <https://docs.python.org/3/library/multiprocessing.html> and <https://docs.python.org/3/library/threading.html> (accessed Oct. 5, 2026)